

Design Document

AI 行为监控系统设计文档

1. 系统概述
2. 系统架构
3. 模块划分
4. 核心数据结构
 - 4.1 ByteTracker 配置
 - 4.2 禁区管理
 - 4.3 轨迹与状态
 - 4.4 日志事件
5. 接口定义
 - 5.1 ZoneManager
 - 5.2 行为评分
 - 5.3 日志记录
 - 5.4 主循环接口
6. 行为分析逻辑
7. 文件输出
8. 扩展点

AI 行为监控系统设计文档

1. 系统概述

该系统基于 YOLOv8 与 ByteTrack 实现多目标实时跟踪和行为监测，重点功能包括：

- 人体检测与跟踪
- 禁区入侵检测
- 奔跑和徘徊行为识别
- 风险评分与事件日志记录
- 视频输出与异常统计

目标是用于安全监控场景，实现自动异常行为报警和记录。

2. 系统架构

AI 智能监控系统 - 系统架构图



3. 模块划分

模块	功能	关键类/函数
视频输入	读取摄像头或视频文件	OpenCV <code>VideoCapture</code>
YOLO检测	检测人体目标	<code>YOLO</code> 类, <code>model(frame)</code>
ByteTrack跟踪	多目标跟踪, ID分配	<code>BYTETracker</code> , <code>BYTETrackerArgs</code>
禁区管理	定义多边形禁区, 判断是否入侵	<code>ZoneManager</code> 类, <code>inside(point)</code> 、 <code>draw(frame)</code>
行为分析	奔跑与徘徊检测	轨迹列表 <code>trajectories</code> , 状态字典 <code>state</code>
风险评分	根据行为计算风险分数	<code>compute_risk(intrusion, running, loitering)</code>
日志系统	保存异常事件	<code>log_event(track_id, event, risk)</code>
视频输出	实时显示或保存视频	OpenCV <code>VideoWriter</code>

4. 核心数据结构

4.1 ByteTracker 配置

	Python
<pre> 1 class BYTETrackerArgs: 2 track_thresh: float # 检测置信度阈值 3 track_buffer: int # 丢失目标保留帧数 4 match_thresh: float # IOU匹配阈值 5 aspect_ratio_thresh: float 6 min_box_area: float 7 mot20: bool </pre>	

4.2 禁区管理

```
1 class ZoneManager:
2     zones: List[List[List[int]]] # 已完成禁区, 多边形点列表
3     current: List[List[int]]     # 当前绘制的禁区
```

4.3 轨迹与状态

```
1 trajectories = { track_id: [(x1, y1), (x2, y2), ...] }
2 state = {
3     track_id: {
4         "speed": [float, ...],
5         "run_frames": int,
6         "loiter_frames": int
7     }
8 }
```

4.4 日志事件

```
1 log = {
2     "id": int(track_id),
3     "event": "INTRUSION" | "RUNNING" | "LOITERING",
4     "risk": int,      # 风险评分0-100
5     "time": str       # 时间戳
6 }
```

5. 接口定义

5.1 ZoneManager

```

1  zone_manager.draw(frame)      # 绘制所有禁区
2  zone_manager.inside(point)    # 判断点是否在禁区内
3  zone_manager.save()          # 保存禁区配置到 zones.json
4  zone_manager.load()          # 加载禁区配置

```

5.2 行为评分

```

1  compute_risk(intrusion, running, loitering) -> int

```

5.3 日志记录

```

1  log_event(track_id, event, risk)

```

5.4 主循环接口

- 输入：摄像头或视频帧 `frame`
- 输出：带标记框的视频帧，事件日志文件 `event_log.jsonl`
- 控制：
 - `q` 退出
 - `s` 保存禁区（注释掉）
 - `c` 清除当前绘制禁区（注释掉）

6. 行为分析逻辑

1. 禁区入侵

- 检测目标中心点是否在任意禁区内

2. 奔跑检测

- 利用速度变化：平均速度 > 15 像素/帧，连续 5 帧判定为奔跑

3. 徘徊检测

- 取最近 30 帧轨迹
- 若活动范围 `dx` 和 `dy` < 60 像素，则连续 10 帧判定为徘徊

4. 风险评分

- 入侵 +50
 - 奔跑 +20
 - 徘徊 +15
 - 最大 100 分
-

7. 文件输出

- 视频输出: `output.avi`
 - 日志文件: `event_log.jsonl` (逐行 JSON)
 - 禁区配置: `zones.json`
-

8. 扩展点

- 多禁区支持与动态调整
- 增加人群密度分析
- 异常报警触发外部系统
- GUI 显示与鼠标绘制禁区 (目前注释掉)